ComplexGitSync 0002.21– Python API

Nicolas Flipo

September 2, 2026

Contents

1 Direct Python Object API

This chapter shows how to manipulate ComplexGitSync objects directly (without the CLI wrapper), including loading a .gts snapshot, reconstructing the runtime registry, mirroring identity into GitTree/GitRepo, and propagating a tag target.

For normal Multi-repo Sync usage, prefer the CLI lifecycle documented in the README and Getting Started guide. The reference test topology is CGSil1 (https://gitlab.com/CGS_test/CGSil1): initialise -> pull -> checkout/add/commit/push -> freeze -> launch_release.

The lower-level object/API lifecycle remains: load(.cgs) -> .gts LOADED -> expand(.cgs|.gts) -> .gts PENDING -> validate(.cgs|.gts) -> .gts READY -> pull(.cgs|.gts) -> checkout(.gts) -> add -> git(tree,"commit",msg) -> git(tree,"push") -> git(tree,"tag",name) -> freeze.

2 Programmatic Project Configuration

Project configuration does not require the CLI or an existing .cgs file. The public facade accepts authoring values non-interactively and delegates normalization, static validation, and optional serialization to cgs_format.py. It performs no Git or network operations.

```
from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
```

```
document = client.configure(
    "GX4G",
    ["codeberg:GX4G/GX4G"],
    output_path="GX4G.cgs",  # optional
)
reference_tree = document.to_git_tree()
```

The CLI commands `configure`, `create-cgs`, and direct `initialise -project/-repo` authoring are adapters over this method.

3 From Empty Registry to READY via `.gts`

```
from pathlib import Path

from ComplexGitSync import GitRepo, GitTree, RefKind, TreeLifecycleState
from ComplexGitSync.orchestre import GtsDocument,
   build_registry_from_gts_document
from ComplexGitSync.git_tree import WorkingGitTree

# 1) Empty registry lifecycle
empty_registry = WorkingGitTree()
assert empty_registry.recompute_tree_state() == TreeLifecycleState.
   UNLOADED

# 2) Load the CGSil1 runtime snapshot (.gts) and rebuild the dependency
   registry
cgshome = Path("../..")
gts_doc = GtsDocument.from_toml(cgshome / ".cgitsync/state/CGSil1.gts")
registry = build_registry_from_gts_document(gts_doc)
assert registry.lifecycle_state == TreeLifecycleState.READY

# 3) Rebuild GitTree/GitRepo identity objects from discovered registry
   entries
tree = GitTree()
for entry in registry.values():
    tree.add_repo(
        GitRepo(
            project_owner_name=entry.project_owner_name or "owner",
            project_name=entry.project_name or entry.name,
            gitprovider=entry.gitprovider,
            access_protocol=entry.access_protocol,
            commit_sha=entry.commit_sha,
        )
    )
```

4 Object-Level Tag Propagation

```
# Set the same tag target across all registry entries (parent -> leaves)
tree.propagate_tag(registry, "v1.2.3")
for entry in registry.values():
    assert entry.target_ref_kind == RefKind.TAG
    assert entry.target_ref_name == "v1.2.3"
```

This object-level flow is the same state model used by the CLI and `ComplexGitSyncClient`, but exposed directly for advanced automation and tests.

5 READY Methods in the Client API

The same lifecycle model is exposed by `ComplexGitSyncClient`. In practice, the workflow is:

- `load(...)` is the step-1 name for direct source loading.
- `expand(...)` is the canonical step-2 name; it runs nested `.cgs` discovery from parents to leaves (recursive).
- `validate(...)` is the canonical step-3 name; it accepts both `.cgs` and `.gts` sources.
- `initialise(...)` bootstraps a tree and must finish in `READY`; `clone(...)` is the direct clone entry point for a `.cgs` source; `pull(...)` resynchronises an existing tree.
- `git(tree, command, *args)` is the unified step-5 interface for `"commit"`, `"push"`, and `"tag"` operations. Each follows leaf-first ordering.
- `freeze(...)` emits the next persisted `.gts` state.

```
from pathlib import Path

from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
cgspath = Path("../..")
cgshome = cgspath / "CGSil1"

# 1. load the CGSil1 test-case .cgs -> .gts LOADED
client.load("../CGSil1.cgs")

# 2. expand(.cgs/.gts) -> .gts PENDING
print(client.expand("../CGSil1.cgs"))

# 3. validate(.cgs/.gts) -> .gts READY
client.validate("../CGSil1.cgs")

# 4. clone(.cgs/.gts)
# Same default as the CLI: output_path defaults to CGSPATH=../..
client.initialise("../CGSil1.cgs")

# Equivalent explicit form:
client.initialise("../CGSil1.cgs", output_path=cgspath)

# Recovery path for partial/stale clone state:
client.clean_initialise_cgs("../CGSil1.cgs", output_path=cgspath)

# Cleanup only:
client.purge_cgs("../CGSil1.cgs", output_path=cgspath)

# 5. READY methods - unified git interface
registry = client.get_dependency_registry()
# Pull uses ROOT -> PARENT -> LEAF; every repository in the tree is a
    plain
# independent clone (there are no gitlinks) and receives its own git
    pull.
# A .gts source must be an actual snapshot path (find the current one
    via
```

```
# RuntimeStateStore.latest_snapshot_for(), the same lookup the CLI uses
    when
# a command's snapshot argument is omitted) -- unlike a .cgs source,
    there is
# no fixed, predictable filename to hard-code here.
client.pull("../CGSil1.cgs")
client.checkout("feature/my-branch")
# Mutations use LEAF -> PARENT -> ROOT.
client.add()
client.git(registry, "commit", "feat: update CGSil1 CGS#1")
client.git(registry, "push")
client.git(registry, "tag", "v1.2.3")

# 6. freeze -> next .gts id
client.freeze("release-2026.05")
```

6 .gitignore Lifecycle Sync and Commit Identity

`initialise_cgs`, `initialise_cgs_document`, `clean_initialise_cgs`, `clean_init`, `restart`, and `pull` (for `.cgs` sources) all accept the same four optional keyword arguments, mirroring the CLI's `-commit-gitignore`, `-force-gitignore-sync`, `-git-user-name`, and `-git-user-email` flags:

```
from ComplexGitSync import ComplexGitSyncClient, MasterConfig

client = ComplexGitSyncClient()
client.initialise_cgs(
    "../CGSil1.cgs",
    output_path=cgspath,
    commit_gitignore=True,        # stage/commit/push each changed .
        gitignore
    force_gitignore_sync=False,   # pull-force fallback for a blocked
        repo
    git_user_name="cgitsync-bot",
    git_user_email="bot@example.com",
)

# The identity above is persisted to CGSHOME/.cgitsync/master.toml, so a
# later call on the same workspace picks it up without repeating it:
client.initialise_cgs("../CGSil1.cgs", output_path=cgspath,
    commit_gitignore=True)
assert MasterConfig.resolve_identity(cgshome, client.git_runner) == (
    "cgitsync-bot",
    "bot@example.com",
)
```

All four keyword arguments default to `False`/`None`: by default, every repository with children (root, or any nested repository that itself has further nested children) is safely pulled and has its `.gitignore` updated, but nothing is staged, committed, or pushed, and the commit identity — were a commit ever made — would be whatever `git config user.name`/`user.email` already resolves to locally. `MasterConfig` (`master.py`) is the class behind the identity override: `configure()` sets it in memory for the current process, `persist(cgshome, ...)` writes it to `CGSHOME/.cgitsync/master.toml` (and also updates the in-memory value), `load(cgshome)` reads a previously persisted override back in, and `resolve_identity(repo_path, git_runner)` returns the `(user_name, user_email)` pair — or `(None, None)` to defer entirely to local git

config. This file is workspace-local configuration, not part of the `.cgs/.gts` project spec, and is preserved (not deleted) by `purge/clean-init`.

7 Forcing a Clone Protocol

`initialise_cgs`, `initialise_cgs_document`, `clean_initialise_cgs`, `clean_init`, `bootstrap`, and `clone_cgs` additionally accept `force_access_protocol`, mirroring the CLI's `-force-protocol` flag on `initialise`, `bootstrap`, and `clean-init` (`restart/pull` do not accept it):

```
# Every repo this run clones -- root siblings and any child discovered
# later from a nested .cgs in a different repo -- is cloned over https,
# regardless of what its own .cgs entry says. No .cgs file is read
# differently or written.
client.initialise_cgs(
    "../CGSil1.cgs",
    output_path=cgspath,
    force_access_protocol="https",
)
```

`"ssh"` or `"https"`; `None` (the default) leaves every entry's own `.cgs`-declared `access_protocol` untouched. The override lives only in memory for the duration of the call – it is applied at the single point every clone's remote URL is built (`ComplexGitSyncClient._build_remote_url`), so it reaches root siblings and nested-discovered children alike without `cgs_format.py` parsing, or any `.cgs` file on disk, ever being touched differently.

8 Repository Discovery: `discover_repos`

`discover_repos(root_dir=None, *, max_depth=5, output=None)` scans a directory for git repositories already checked out on disk and drafts a `.cgs` from what it finds — the entry point for adopting a project that exists but has no specification yet:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Read-only scan - nothing is cloned, fetched, or modified
report = client.discover_repos("/path/to/checkout")
for repo in report.repos:
    print(repo.relative_path, repo.identifier, repo.branch)

for warning in report.warnings:
    print("unresolved:", warning)

# Write the draft out (same call, with an output path)
report = client.discover_repos("/path/to/checkout", output="draft.cgs")
```

The method returns a `DiscoverReport` dataclass with fields `root` (the scanned directory), `repos` (a tuple of `DiscoveredRepo`), `cgs_entries` (authoring-form repo tables ready for `configure()`), `warnings`, and `project_name`. Each `DiscoveredRepo` carries `relative_path`, `absolute_path`, `remote_url`, `identifier`, `branch`, and `has_cgs`.

Repository addresses are resolved through the existing `parse_repo_id()` grammar; a repository with no `origin` or an unrecognised provider is reported in `warnings` and omitted from `cgs_entries` rather than guessed at. Passing `output` raises `GitSyncError` when nothing resolvable was found, so an empty draft is never written.

9 Submodule Migration: `import_submodules`

`import_submodules(repo_root, *, apply=False, output=None)` converts git submodules in a local repository to plain ComplexGitSync nested clones:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Dry run - inspect without changing anything
report = client.import_submodules("/path/to/repo")
for sub in report.submodules:
    print(sub.name, sub.path, sub.url)

# Apply - convert and optionally emit a .cgs snippet
report = client.import_submodules(
    "/path/to/repo",
    apply=True,
    output="imported_submodules.cgs",
)
print(report.converted)   # tuple of converted submodule names
```

The method returns an `ImportSubmodulesReport` dataclass with fields `submodules` (all entries found), `applied` (whether the conversion ran), `converted` (names of converted submodules), and `cgs_entries` (authoring-form repo tables ready for `configure()`). When `apply=False` (the default), no files are modified. When `apply=True`, each submodule is removed from the index (`git rm -cached`), its stanza is removed from `.gitmodules`, and the parent's `.gitignore` is updated to ignore the child path. The staged changes should then be reviewed and committed manually.

10 Register Integrity: `verify`

`verify(cgshome, *, repair=False)` checks the hash-chained `.cgitsync/lgr` register under `cgshome` for tamper-evidence:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
report = client.verify("/path/to/CGSHOME")

if report.is_clean:
    print("register is clean")
else:
    for seq, finding, detail in report.findings:
        print(seq, finding.name, detail)

# Repair a stale HEAD cache (never rewrites or deletes an entry)
client.verify("/path/to/CGSHOME", repair=True)
```

Returns an `integrity.VerificationReport` (re-exported from the package root) whose `findings` list holds `(seq, Finding, detail)` tuples and whose `is_clean` property is `True` when the list is empty. A register with no entries yet is reported clean — this is the common case today, since no command writes to this per-entry format yet (the CLI's `verify` command is the enforcement instrument; the migration of `SyncLedger`'s own writes to this format is a separate, future piece of work). `Finding` enumerates `BROKEN_LINK`, `BAD_ENTRY_HASH`, `SEQ_GAP`, `SEQ_DUPLICATE`, and `HEAD_STALE` for what this method actually checks (chain linkage and the `HEAD` cache); its remaining three members (`MISSING_STATE`, `ORPHAN_STATE`, `STATE_DIGEST_MISMATCH`)

are reserved for a future store-level verification pass that cross-references entries against the actual `state(<hash>)_n/` directories on disk.

With `repair=True`, a stale `HEAD` cache is corrected in place. Register entries are never rewritten or deleted by this method, repair or not — a broken chain is reported, not silently healed.